BẢN HƯỚNG DẪN KIẾN TRÚC & PHÂN CÔNG NHIỆM VỤ BTL

Môn học: Nhập môn Học máy & Khai phá dữ liệu (IT3190) **Nhóm trưởng:** Hải Nam

🎯 **Đề tài:** Phân loại cảm xúc văn bản tiếng Việt (UIT-VSMEC) kết hợp LLM phản hồi thông minh.
🔗 **Luồng dữ liệu qua 5 khối:** Raw Text $\rightarrow$ `Khối Tiền xử lý (Hân)` $\rightarrow$ `Khối TF-IDF & Linear ML (Khôi)` $\rightarrow$ `Khối Cây quyết định (Kiên)` & `Khối Đánh giá (Hoàng Anh)` $\rightarrow$ `Khối Điều phối, LLM & Web App (Nam)`.

I. BẢNG PHÂN CÔNG KHỐI CHỨC NĂNG HỆ THỐNG

Thành viên & MSSV	Khối chức năng phụ trách (Subsystem)	Thư mục sở hữu & Trách nhiệm đầu ra	Nhánh Git
Bùi Mai Hân 202400041	DATA PREPROCESSING SUBSYSTEM **Khối Tiền xử lý dữ liệu**	`source/data_process/preprocessing/` • Class `TextCleaner` & `teencode_dict.py` • Dữ liệu sạch `data/clean_data/` • Biểu đồ phân bố `docs/figures/`	`feature/data-preprocessing`
Vũ Đình Khôi 20235124	FEATURE & LINEAR ML SUBSYSTEM **Khối Đặc trưng & Mô hình Tuyến tính**	`featuring/` & `classifier/algorithms/linear/` • Lớp trừu tượng `BaseSentimentClassifier` • Class `TfidfFeatureExtractor` • 3 mô hình: Naive Bayes, LogReg, LinearSVC • File lưu `best_model.joblib`	`feature/ml-linear-models`
Vũ Trung Kiên 202416253	TREE & ENSEMBLE SUBSYSTEM **Khối Mô hình Cây & Ensemble**	`classifier/algorithms/tree/` • Class `DecisionTreeModel` • Class `RandomForestModel` • Notebook thực nghiệm đo đạc Train Time & F1	`feature/ml-tree-models`
Nguyễn Hoàng Anh 202416121	EVALUATION ANALYTICS SUBSYSTEM **Khối Đánh giá & Phân tích Chất lượng**	`classifier/evaluator.py` • Class `ModelEvaluator` • Bảng Benchmark so sánh 5 mô hình • Ảnh Heatmap Confusion Matrix • Phân tích 5 câu đoán sai cho Slide	`feature/model-evaluation`
Hải Nam Nhóm trưởng	CORE ORCHESTRATION & FULL APP **Khối Điều phối, LLM & Web App**	`services/`, `api/`, `ui/`, `deploy/` • Tầng điều phối `OrchestratorService` • Module kết nối Google Gemini CSKH • FastAPI Server, Streamlit Web UI, Docker	`main`

⚠️ **3 NGUYÊN TẮC BẢO VỆ HỆ THỐNG (CHỐNG CONFLICT):**
1. **Ranh giới thư mục:** Mỗi bạn chỉ code trong thư mục được phân công ở trên. Tuyệt đối không sửa file của nhau.
2. **Tuân thủ OOP & Interface:** Mọi mô hình phải kế thừa lớp `BaseSentimentClassifier` để hệ thống gọi đồng nhất.
3. **Không hardcode path:** Dùng `pathlib.Path` lấy đường dẫn tương đối, không dùng đường dẫn tuyệt đối cá nhân.

II. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC CHO TỪNG THÀNH VIÊN

1. Bùi Mai Hân — Khối Tiền xử lý dữ liệu (Data Preprocessing)

NHÁNH: FEATURE/DATA-PREPROCESSING

Vị trí & Ý nghĩa: Cửa ngõ đầu vào chuẩn hóa câu bình luận mạng xã hội nhiều teencode/icon.
Input: `data/raw_data/*.json` $\rightarrow$ **Output bàn giao:** `data/clean_data/*.json` & hàm `clean_text()`.

Bước 1: Tạo từ điển Teencode (`source/data_process/preprocessing/teencode_dict.py`)

Định nghĩa dictionary Python khoảng 30-50 từ teencode thông dụng: `TEENCODE_DICT = {"ko": "không", "dc": "được", "vs": "với", "mik": "mình", "lm": "làm"...}`.

Bước 2: Cài đặt Class `TextCleaner` (`source/data_process/preprocessing/text_cleaner.py`)

```
class TextCleaner:
    def __init__(self, teencode_dict: dict = None):
        self.teencode = teencode_dict or TEENCODE_DICT

    def clean_text(self, text: str) → str:
        text = text.lower()                                  # 1. Chữ thường
        text = re.sub(r'https?://\S+|www\.\S+', '', text)   # 2. Xóa URL
        text = re.sub(r'[^\w\s]', ' ', text)                # 3. Xóa ký tự đặc biệt thừa
        words = [self.teencode.get(w, w) for w in text.split()] # 4. Thay teencode
        return ' '.join(words).strip()                       # 5. Xóa khoảng trắng thừa

    def clean_dataset(self, input_path: Path, output_path: Path):
        # Đọc file raw JSON, làm sạch trường Sentence, ghi ra output_path JSON
```

Bước 3: Tự kiểm thử & Chạy xuất dữ liệu

Viết `if __name__ == "__main__"`: kiểm tra thử 1 câu. Sau đó gọi `clean_dataset` tạo ra 3 file sạch trong `data/clean_data/`.

Bước 4: Vẽ biểu đồ EDA & Bàn giao

Mở `notebooks/01_eda_and_visualizations.ipynb`, vẽ biểu đồ phân bố 7 nhãn lưu vào `docs/figures/`. Đẩy code lên nhánh của mình.

2. Vũ Đình Khôi — Khối Đặc trưng & Mô hình Tuyến tính (Linear ML)

NHÁNH: FEATURE/ML-LINEAR-MODELS

Vị trí & Ý nghĩa: Trái tim thuật toán cốt lõi. Chuyển chữ thành vector TF-IDF và huấn luyện các mô hình tuyến tính mạnh nhất.
Input: Dữ liệu sạch từ Hân $\rightarrow$ **Output bàn giao:** `tfidf_vectorizer.joblib`, `best_model.joblib`, class trừu tượng cho Kiên.

Bước 1: Định nghĩa Lớp trừu tượng (`source/model/classifier/base_classifier.py`)

```
from abc import ABC, abstractmethod

class BaseSentimentClassifier(ABC):
    @abstractmethod
    def train(self, X_train, y_train): pass

    @abstractmethod
    def predict(self, X_test) → np.ndarray: pass

    @abstractmethod
    def predict_proba(self, X_test) → np.ndarray: pass  # Trả về % xác suất cho Web

    @abstractmethod
    def save(self, filepath: str): pass
```

Bước 2: Xây dựng Class `TfidfFeatureExtractor` (`source/data_process/featuring/tfidf_extractor.py`)

Dùng `TfidfVectorizer(max_features=5000, ngram_range=(1, 2))`, cài đặt `fit_transform`, `transform`, `save`, `load`.

Bước 3: Cài đặt 3 mô hình tuyến tính (kế thừa BaseSentimentClassifier)

Tạo `MultinomialNBClassifier`, `LogisticRegressionClassifier`, `LinearSVCClassifier` (dùng `CalibratedClassifierCV(LinearSVC())` để có xác suất).

Bước 4: Huấn luyện, Lưu model vô địch & Bàn giao

Chạy notebook `02_linear_models.ipynb`; lưu model F1 cao nhất vào `saved_models/best_model.joblib`; chia sẻ vectorizer cho Kiên và số liệu cho Hoàng Anh.

3. Vũ Trung Kiên — Khối Mô hình Cây & Ensemble (Tree ML)

NHÁNH: FEATURE/ML-TREE-MODELS

Vị trí & Ý nghĩa: Nghiên cứu thực nghiệm mô hình dạng cây phi tuyến, chứng minh tính phù hợp so với mô hình tuyến tính trên văn bản.
Input: Vector TF-IDF từ Khôi $\rightarrow$ **Output bàn giao:** Class `tree_models.py` và bảng đo thời gian/F1.

Bước 1: Cài đặt Class mô hình Cây kế thừa lớp cơ sở (`algorithms/tree_models.py`)

```
from source.model.classifier.base_classifier import BaseSentimentClassifier

class DecisionTreeModel(BaseSentimentClassifier):
    def __init__(self, max_depth=20, random_state=42):
        self.model = DecisionTreeClassifier(max_depth=max_depth, random_state=random_state)
    def train(self, X_train, y_train): self.model.fit(X_train, y_train)
    def predict(self, X_test): return self.model.predict(X_test)
    def predict_proba(self, X_test): return self.model.predict_proba(X_test)
    def save(self, filepath: str): joblib.dump(self.model, filepath)

class RandomForestModel(BaseSentimentClassifier):
    # Cài đặt tương tự với RandomForestClassifier(n_estimators=100, max_depth=20)
```

Bước 2: Chạy thực nghiệm & Đo đạc thời gian huấn luyện

Mở `notebooks/03_tree_models.ipynb`, nạp TF-IDF từ Khôi, huấn luyện 2 mô hình. Dùng `time.time()` bấm giờ thời gian train của Random Forest.

Bước 3: Rút ra nhận xét học thuật & Bàn giao

Ghi nhận Macro-F1, Accuracy, Train Time gửi qua Hoàng Anh; phân tích vì sao Cây quyết định dễ bị quá khớp (overfit) trên ma trận thưa TF-IDF.

4. Nguyễn Hoàng Anh — Khối Đánh giá & Phân tích Chất lượng (Evaluation Analytics)

NHÁNH: FEATURE/MODEL-EVALUATION

Vị trí & Ý nghĩa: Kiểm định chất lượng độc lập, tổng hợp so sánh 5 mô hình, vẽ ma trận nhầm lẫn và mổ xẻ nguyên nhân đoán sai.
Input: Kết quả dự đoán 5 mô hình từ Khôi & Kiên $\rightarrow$ **Output bàn giao:** Benchmark Table, ảnh Confusion Matrix, 5 câu phân tích lỗi cho Slide.

Bước 1: Cài đặt Class `ModelEvaluator` (`source/model/classifier/evaluator.py`)

```
class ModelEvaluator:
    @staticmethod
    def compute_metrics(y_true, y_pred) → dict:
        return {"accuracy": accuracy_score(y_true, y_pred), "macro_f1": f1_score(y_true, y_pred, av

    @staticmethod
    def plot_and_save_confusion_matrix(y_true, y_pred, labels: list, save_path: str):
        # Vẽ Heatmap seaborn confusion matrix và lưu ảnh 300 DPI ra save_path
```

Bước 2: Lập bảng so sánh Benchmark đối đầu 5 mô hình

Mở `notebooks/04_benchmarking_and_errors.ipynb`, thu thập số liệu Khôi & Kiên điền vào bảng so sánh: Naive Bayes, LogReg, LinearSVC, Decision Tree, Random Forest.

Bước 3: Xuất biểu đồ Ma trận nhầm lẫn & Phân tích lỗi (Error Analysis)

Chạy xuất ảnh `confusion_matrix_best.png` vào `docs/figures/`. Lọc ra 3-5 câu mô hình đoán sai (câu mỉa mai, teencode lạ) phân tích lý do để đưa vào Slide.

5. Hải Nam (Nhóm trưởng) — Khối Điều phối, LLM & Web App

NHÁNH: MAIN

Nhiệm vụ trọng tâm:

• **Khối LLM (`source/model/llm/`):** Tích hợp Google Gemini sinh phản hồi CSKH xoa dịu/tri ân theo cảm xúc.

• **Khối Điều phối (`source/services/`):** Nối luồng `TextCleaner` $\rightarrow$ `TF-IDF` $\rightarrow$ `LinearSVC` $\rightarrow$ `LLM Gemini`.

• **Khối API & Web UI:** Dựng **FastAPI Backend** & **Streamlit Web App** trực quan.

• **Đóng gói:** Docker Compose, hoàn thiện Slide và Báo cáo BTL trong `docs/report/`.

III. CẨM NANG GIT & GITHUB TOÀN TẬP (DÀNH CHO NGƯỜI CHƯA BIẾT GÌ)

💡 **HIỂU BẢN CHẤT TRONG 30 GIÂY (MÔ HÌNH 3 TRẠM):**

```
[File trên máy bạn] —(git add .)—> [Hòm thư chờ gửi] —(git commit)—> [Két sắt máy bạn]
—(git push)—> [GitHub chung] (Working Directory) (Staging Area) (Local Repo) (Remote Repo)
```

- `git add .` : Nhặt các file vừa sửa/tạo mới bỏ vào "Hòm thư chờ gửi".
- `git commit -m "tin_nhan"` : Niêm phong hòm thư, ghi nhãn công việc vừa làm.
- `git push` : Gửi hòm thư đó lên trang web GitHub của nhóm.

Giai đoạn 1: Cài đặt và Tải dự án về máy (Chỉ làm duy nhất 1 lần đầu)

Bước 1.1: Kiểm tra Git trên máy tính

Mở Terminal / PowerShell gõ: `git --version` (nếu báo lỗi chưa nhận lệnh thì vào git-scm.com tải về cài đặt).

Bước 1.2: Khai báo danh tính của bạn (Bắt buộc để GitHub ghi nhận bạn đóng góp)

```
git config --global user.name "Nguyen Van A"
git config --global user.email "nguyenvana@gmail.com"
```

Bước 1.3: Tải mã nguồn dự án về máy

Mở VS Code tại thư mục muốn lưu bài tập, mở Terminal và gõ:

```
git clone <LINK_REPO_GITHUB_DO_NAM_GUI>
cd IT3190_Project
```

Bước 1.4: Chuyển sang ĐÚNG nhánh được phân công (RẤT QUAN TRỌNG)

```
# Mai Hân:    git checkout feature/data-preprocessing
# Đình Khôi:  git checkout feature/ml-linear-models
# Trung Kiên: git checkout feature/ml-tree-models
# Hoàng Anh:  git checkout feature/model-evaluation
```

Gõ `git branch` để kiểm tra: Thấy dấu sao màu xanh * ở đúng nhánh của bạn là chuẩn!

Giai đoạn 2: Vòng lặp làm việc hàng ngày (Daily Workflow)

Mỗi khi ngồi vào bàn học để code, bạn thực hiện đúng 5 bước sau:

Bước 2.1: Cập nhật code mới nhất từ Nhóm trưởng (Tránh bị lỗi code cũ)

```
git pull origin main
```

Bước 2.2: Tiến hành code trong đúng thư mục sở hữu của mình

Bước 2.3: Xem danh sách các file bạn vừa chỉnh sửa

```
git status
```

Bước 2.4: Đóng gói các thay đổi (Add & Commit)

```
git add .
git commit -m "feat: hoan thanh xong phan duoc giao"
```

Bước 2.5: Đẩy code lên GitHub

```
git push origin <ten-nhanh-cua-ban>
```

Giai đoạn 3: Tạo Pull Request (PR) trên web GitHub để Nhóm trưởng duyệt

1. Mở trình duyệt web vào link GitHub của nhóm.
2. Bạn sẽ thấy thanh thông báo màu vàng nổi lên: **"feature/... had recent pushes"**. Bấm nút màu xanh **Compare & pull request**.
3. Kiểm tra mũi tên trỏ về: `base: main` $\leftarrow$ `compare: <nhanh-cua-ban>`.
4. Ghi tiêu đề ngắn gọn công việc và bấm nút xanh lớn **Create pull request**.
5. Nhắn tin nhóm Zalo: *"Nam ơi tớ tạo PR rồi, bạn vào review và merge giúp tớ nhé!"*.

Giai đoạn 4: "Cẩm nang cấp cứu" khi gặp sự cố Git

Tình huống / Sự cố	Nguyên nhân	Cách xử lý an toàn
Lỡ sửa nhầm file của người khác	Mở nhầm file không thuộc thư mục của mình	Gõ: `git restore <duong_dan_file>` để hoàn tác
Gõ `git push` bị báo lỗi `[rejected]`	Trên GitHub có code mới hơn mà máy bạn chưa tải về	Gõ `git pull origin <nhanh_ban>` rồi push lại
Không biết mình đang ở nhánh nào	Quên tên nhánh hiện tại	Gõ `git branch` xem nhánh có dấu sao `*`
Báo lỗi chữ đỏ CONFLICT hoặc màn hình lạ	Xung đột nội dung giữa 2 nhánh	**DỪNG LẠI NGAY!** Không gõ bừa. Chụp màn hình gửi Nhóm trưởng Hải Nam hỗ trợ gỡ rối ngay!